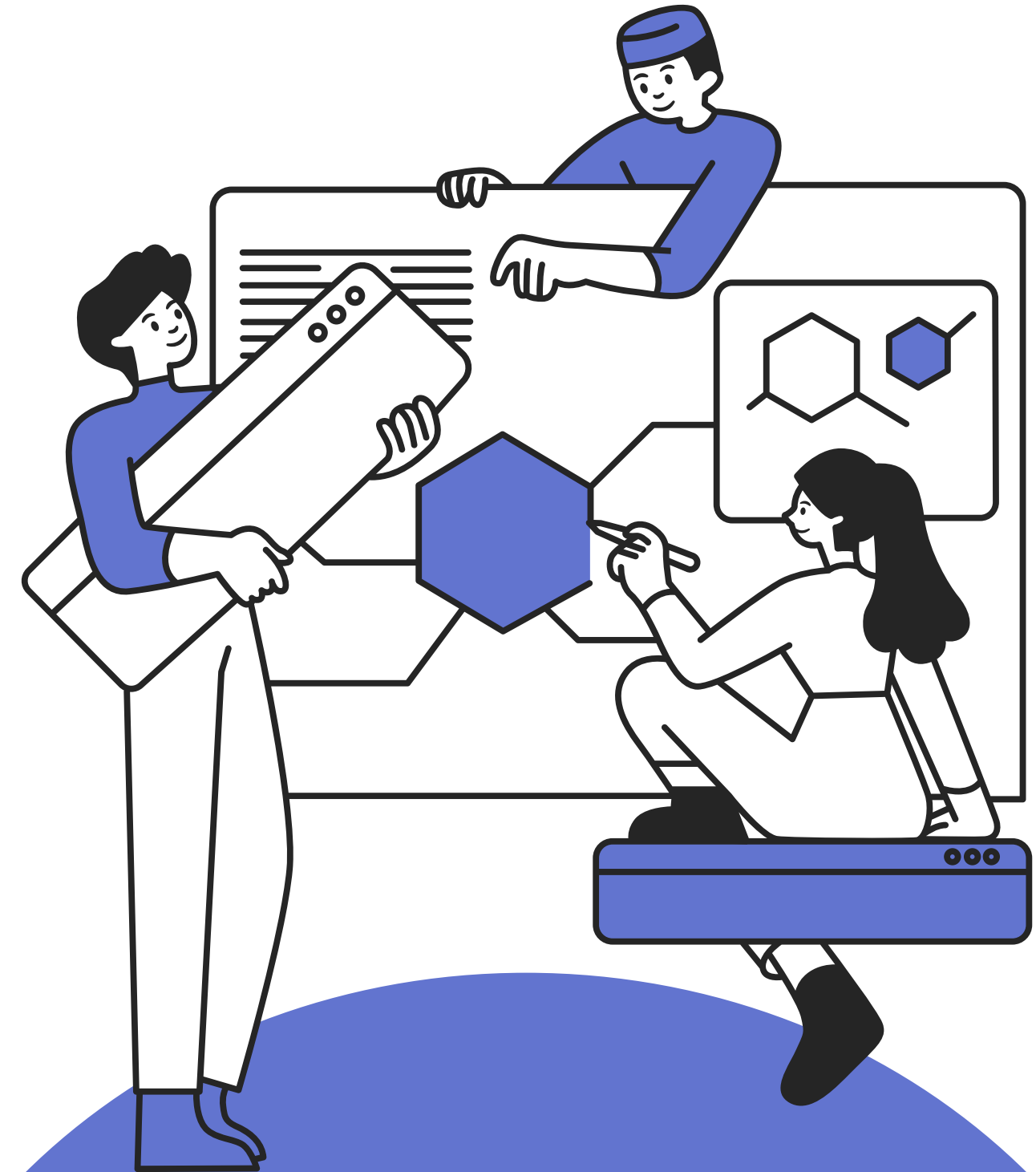


고객 및 상품 관리 시스템 프로젝트

4조. 결제는 제가 할게요
발표자. 정민교



CONTENTS

- 1 프로젝트 소개(Intro)
- 2 사용자 & 문제정의
- 3 팀 운영 & 협업 방식
- 4 일정 계획(Milestone)
- 5 시스템 설계(Architecture & ERD)
- 6 구현 결과
- 7 핵심 기능 시연
- 8 트러블슈팅 · 회고 · Q&A

01

프로젝트 소개

이커머스 백오피스 관리 시스템

고객 · 상품 · 주문 데이터를 운영자가 효율적으로 관리할 수 있도록 관리자 중심의 백오피스 시스템을 설계·구현

Spring Boot + JPA 기반으로 REST API를 구현하였으며, CRUD, 검색·정렬·페이징, 세션 인증 기능까지 필수 기능 범위 내에서 완성

ACME 어드민

고객관리

주문목록

상품관리

판매자 정보

결제내역

고객관리

이탈 위험 고객

아이디

이름

조회

☐	ID	이름	전화번호
☐	8837	김수정	010-1111-2
☐	8836	박찬기	010-1123-4
☐	8835	장비오	010-9988-9
☐	8834	John Kane	010-7777-2

02 사용자 & 문제정의



이커머스 서비스에서는
주문 · 재고 · 매출 데이터가 지속적으로 누적되며
각 데이터는 실제 거래 기록과 직결됨



데이터가 많아질수록
검색, 정렬, 상태 관리의 중요성이 증가
→ 재고 수량과 주문 상태가 일치하지 않을 경우
매출 집계 오류 · 환불 분쟁 · 정산 불일치 발생



체계적인 백오피스 시스템이 없다면
수기 관리 의존 증가 및 데이터 정합성 붕괴
→ 운영 리스크 및 회계 대응 어려움

03 사용자 & 문제정의

운영 관리자

- 상품 등록 및 재고 관리
- 주문 상태 변경 담당
- 빠른 검색과 정렬 기능 필요

CS 관리자

- 고객 문의 응대
- 대리 주문 및 취소 처리
- 고객 상태 및 주문 이력 확인 필요

슈퍼 관리자

- 관리자 승인 / 거부
- 계정 관리 및 통제
- 시스템 안정성 유지

04

사용자 & 문제정의



관리자 중심 설계



데이터 기반 관리



재고 - 주문 정합성



승인 기반 통제

05 팀 운영 & 협업 방식

팀 운영 역할

역할	이름	담당 task
리더	정채림	총괄, 일정 지연 시 인지
부리더	이재민, 이지민, 신현민	전역 서포터
발표자료 제작 담당	이지민, 이지혜	발표자료 제작, 발표 리허설 주도
기록 담당	정민교, 신현민	회의록 정리 / Readme 수합 및 관리

06 팀 운영 & 협업 방식

도메인 담당

역할	이름	담당 task
고객 관리	이지혜, 이재민, 정민교	고객 관리(조회/수정/삭제/상태 변경)
상품 관리	신현민	상품 관리(등록/조회/수정/삭제/재고변경/상태변경 + 재고 기반 자동 전환)
주문(R)	정민교	주문 목록 조회 (검색, 정렬, 페이징), 주문 상세 조회
주문(CUD)	이재민	주문 생성(재고 차감, 상태 변경), 취소 트랜잭션/동시성 제어 핵심 담당
관리자/인증	이지민	관리자 가입/승인 프로세스, 세션 로그인/로그아웃 구현
공통/QA	정채림	프로젝트 초기 세팅 (패키지, Git, S.A 문서) ,공통 Response/Exception 핸들러 구현 전체 코드 병합 및 QA (테스트) 총괄

07 팀 운영 & 협업 방식

1) Git 전략

 **first-prj-be** Public

develop 8 Branches 0 Tags

Switch branches/tags

Branches

Tags

✓ develop default

feat/Product개발#14

feat/admin

feat/customer-detail#35

feat/customer-paging#51

fix/Category-restore#50

main

test

[View all branches](#)

team4th-project-BE Add status update Insights Workflows ...

Backlog Board Current iteration Roadmap My items + New view

Filter by keyword or by field View

In progress 2 / 12 Estimate: 0

This is actively being worked on

first-prj-be #35
[Feat] 고객 상세 조회
#52

first-prj-be #51
[Feat] 고객 리스트 조회 페이징 기능 추가

+ Add item

Review 3 Estimate: 0

first-prj-be #34
feat: add Customer entity

first-prj-be #52
Feat/customer detail#35

first-prj-be #54
Feat/customer paging#51

+ Add item

Done 24 Estimate: 0

This has been completed

first-prj-be #13
[Feat] order 관련 controller, repository, service
3 Layer Architecture 설계하기
#17

first-prj-be #53
fix: 카테고리 복원 기능 및 주문 취소 기능의 결함 수정

first-prj-be #48
hotfix: 세션 인증 객체를 공통화하고 필드명 오류와 불
일치 등 실행 오류 수정 완료

first-prj-be #50
[Fix] 카테고리 복원 기능 및 삭제된 카테고리 조회 기능
수정
#53

+ Add item

08 팀 운영 & 협업 방식

2) 코드 리뷰

☐	feat: 고객 정보 수정, 상태 변경, 삭제 3레이어 아키텍처 구현 #43 by DuJoneKoo was merged 2 days ago · Approved 4 tasks
☐	feat: 관리자 수정·역할·상태 변경 및 삭제 API 구현 (관리자 관리 write 기능) #42 by jilimni was merged 2 days ago · Approved
☐	주문 상세조회 기능 개발 및 공통 예러코드 수정 #39 by DuJoneKoo was merged 2 days ago · Approved 3 tasks
☐	feat: 관리자 목록/상세 조회 API 구현 / 검색,정렬,필터,페이징 지원 #38 by jilimni was merged 2 days ago · Approved
☐	Feat/admin fix #37 by jilimni was merged 2 days ago · Approved 1 task
☐	feat: add Customer entity feat #34 by singasong219 was merged 2 days ago · Approved
☐	feat: 관리자 내 정보 조회/수정 및 비밀번호 변경 API 구현 #33 by jilimni was merged 2 days ago · Approved
☐	feat:상품 서비스 제외 모든 기능 구현,상품 조회/검색/재고 상태 자동 갱신 기능 추가 feat #32 by hyeonmin02 was merged yesterday · Approved 6 tasks 3 상품 (CRUD) 개발
☐	feat: 주문 엔티티 수정 및 CUD 구현 완료 bug feat #31 by CheatisKey was merged 2 days ago · Approved 3 주문 CUD 개발 (...)
☐	feat: 관리자 로그인/로그아웃 API 추가 / POST /admins/login,logout 구현 #30 by jilimni was merged 5 days ago · Approved
☐	feat: 관리자 로그인/로그아웃 서비스 구현 / 세션 저장 및 상태별 로그인 차단 #29 by jilimni was merged 5 days ago · Approved



CheatisKey commented [yesterday](#)

Member ...

💡 개요

- 카테고리 Entity 수정 및 CRUD 구조 설계

🔧 작업 내용

- 카테고리를 계층형으로 구성하면서 양방향 매핑을 채택
- 순환참조가 걸릴 수 있다는 점을 고려하고 편의 메서드 방법 파악
- SoftDelete를 사용하면서 자식 카테고리가 고아가 됐을 때의 상황을 고민 및 설계

💬 리뷰 포인트

- 처음에 생성하는 카테고리는 부모가 없다. 그때는 어떻게 처리하였는가?
- 부모 카테고리가 삭제된다면, 어떤 과정을 거치게 되는가?
- 코드 컨벤션을 지켰는가?
- 잘못된 설계나, DDD 및 OOP를 준수하지 못하였는가?
- Closes: [\[Feat\] 카테고리 엔티티 및 CRUD 구현하기 #41](#)



Reviewers



Assignees



Labels

feat

Projects

team4th-proje
Status: Done

Milestone

카테고리 CRUD 개발 (이

Development

Successfully merging these issues.

[\[Feat\] 카테고리 엔티](#)

09 팀 운영 & 협업 방식

2) 코드 리뷰

jiiimni approved these changes yesterday

jiiimni left a comment

@transactional(readOnly = true) 기본 + 쓰기 메서드에만 **@transactional** 붙인 구조 곳곳 부모 카테고리 null 허용(최상위 카테고리) 설계 자체는 자연스럽게 DTO에서도 parent null 처리 잘했네요 existsByName으로 중복 카테고리명 방지 로직 들어간 것, findAllWithParent(Pageable)로 목록에서 parent를 미리 fetch하는 것도 좋습니다

src/main/java/com/example/commercepilot/category/controller/CategoryController.java

jiiimni started a review

src/main/java/com/example/commercepilot/category/service/CategoryService.java

```
83 + }
84 +
85 + public CategoryDeleteResponse getDeletedCategories() {
86 +     return null;
```

jiiimni

CategoryService.getDeletedCategories()가 return null; 미구현이라 호출되면 문제가 생겨서 확인부탁드립니다

fryzke left a comment

전체적으로 레이어드 아키텍처 잘 지켜진 코드였습니다. 많이 고민하고 코드를 작성하신게 보여요. 수고 많으셨습니다. 코멘트 단 부분도 한 번 확인해주세요.

src/main/java/com/example/commercepilot/category/dto/response/CategoryDeleteResponse.java

```
8 + @Builder
9 + public record CategoryDeleteResponse(
10 +     Long id,
11 +     String name,
```

fryzke yesterday

category 생성 응답에서는 categoryName을 쓰고 있는데 여기서 name을 쓰고 있습니다. 따로 그렇게 설정하신 이유가 있을까요? 변수명을 통일한다면 더 좋은 코드가 될 거 같습니다

CheatIsKey yesterday

넵. 변수명을 일관성있게 가져가겠습니다.

Reply...

10 팀 운영 & 협업 방식

2) 코드 리뷰

src/main/java/com/example/commercepilot/category/controller/CategoryController.java

Outdated Hide resolved

57 + }

58 +

59 + @GetMapping("/deleted")

60 + public ResponseEntity<ApiResponse<CategoryDeleteResponse>> deleteCategory

jiiimni yesterday

Member ...

이 메서드는 삭제된 목록 조회인데 이름이 deleteCategory라서 아래 DeleteMapping이랑 혼동이 생길 거 같아요
컨트롤러에 deleteCategory가 2개

jiiimni yesterday

Member ...

삭제된 목록을 조회하는 API인데 리턴 타입이 단건처럼 되어있어서
ApiResponse<List> 또는 ApiResponse<Page<...>>로 바꾸는 건 어떨까요

CheatIsKey yesterday

Member Author ...

감사합니다. List로 여러 결과를 받을 수 있도록 수정하겠습니다.
메서드명도 수정하겠습니다.

Reply...

src/main/java/com/example/commercepilot/category/dto/response/CategoryListResponse.java

Outdated Hide resolved

Comment on lines 25 to 26

25 + .createdAt(category.getCreatedAt())

26 + .modifiedAt(category.getModifiedAt())

fryzke yesterday

Member ...

리스트 목록을 조회할 때 생성시간과 수정시간을 넘겨줄 필요가 있는지도 고려해보면 좋을 거 같습니다.
꼭 필요한 정보인가?를 한 번 생각하고, 줄여나가면 응답 속도를 향상시킬 수 있습니다.

CheatIsKey yesterday

Member Author ...

기본적인 형식으로만 구성하다가, 성능적인 이슈는 생각하지 않았네요.
알려주셔서 감사합니다.

Reply...

Unresolve conversation

fryzke marked this conversation as resolved.

11 팀 운영 & 협업 방식

3) 스크럼 & 문서화

first-prj-docsPublic

develop6 Branches0 Tags

Go to file

t+<>

fryzke Merge pull request #4 from sprt-team4th-project/DuJoneKo...630680e · 2 hours ago10 C

README.md	first commit	la
mitting-note-260220(1).md	Create mitting-note-260220(1).md	5 d
mitting-note-260220(2).md	Add daily scrum meeting minutes for 2026-02-20	5 d
mitting-note-260223.md	Add daily scrum meeting minutes for 2026-02-23	2 d
mitting-note-260224.md	Revise meeting notes with updated agenda and ...	2 h

README

4조 프로젝트 문서

viewCodeBlame

Daily Scrum Meeting Minutes

Date (일시): 2026.02.20 (데일리 스크럼)

Location (장소): ZEP

Recorder (작성자): 정민교

Attendees (참석자): 정채림(팀장), 신현민, 이지민, 정민교, 이지혜, 이재민

1. Meeting Agenda (회의 안건)

- Project Status Sync: 프로젝트 진행 상황 공유
- Weekend Work Plan: 주말 ZEP 참여 인원 및 작업 계획 공유
- Next Steps & Role Allocation: 향후 일정 및 역할 분담

2. Key Discussions (주요 논의 사항)

- 정민교: - Order 도메인의 3-Layer Architecture 구성 및 GET 관련 API 명세서 작성
 - 현재 목록 조회(List)만 구현 완료. 이재민 님의 Order CUD(생성/수정/삭제) 구현이 완료되면 단건 조회 API 추가 구현 예정
- 이지혜: - Customer (고객 관리) Entity 설계 및 생성 예정 (내일 저녁까지 완료 목표)
- 이재민: - 팀원 Entity 코드 리뷰 및 Fix 진행
 - Category 계층형 구조(대/중/소분류) 설계를 위한 연관관계 매핑 진행
 - Order 도메인의 3-Layer Architecture 뼈대 구축 및 POST (주문 생성) 로직 완벽 구현
- 정채림 (팀장): - 전체 프로젝트 진행 상황 모니터링 및 Git Merge 수행
 - 블로커(Blocker) 발생 및 개발 지연 시 즉각적인 팀 내 공유 당부
- 신현민: - Product 도메인 기본 스키폴딩(3-Layer, Enum, Entity) 진행 중 (WIP)

12 일정 계획

기획 & SA 작성

- 요구사항 분석
- ERD / API 명세 초안 작성
- 팀 규칙 및 Git 전략 수립

도메인별 기능 구현

- 관리자 / 고객 / 상품 / 주문 CRUD
- 검색, 정렬, 페이징 구현
- 세션 기반 로그인 처리

핵심 로직 구현

- 주문 생성 시 재고 차감
- 주문 취소 시 재고 복구
- 상태 자동 전환 처리

통합 & QA

- 기능 통합 테스트
- 예외 처리 및 공통 Response 정리
- 버그 수정 및 코드 정리

발표 준비

- PPT 제작
- 시연 시나리오 정리

13 일정 계획

Development Timeline - 고객 및 상품관리 프로젝트(2026.02.19~02.26)							
PROCESS	02-19	02-20	02-21	02-22	02-23	02-24	02-25
ERD 구성							
역할분담							
API 명세서 제작							
프로젝트 세팅/구성							
상품 개발							
관리자 개발							
주문 개발							
고객 개발							
카테고리 개발							
인증 / 세션 개발							
테스트							

14

시스템 설계 (Architecture & ERD & 개념 ERD)

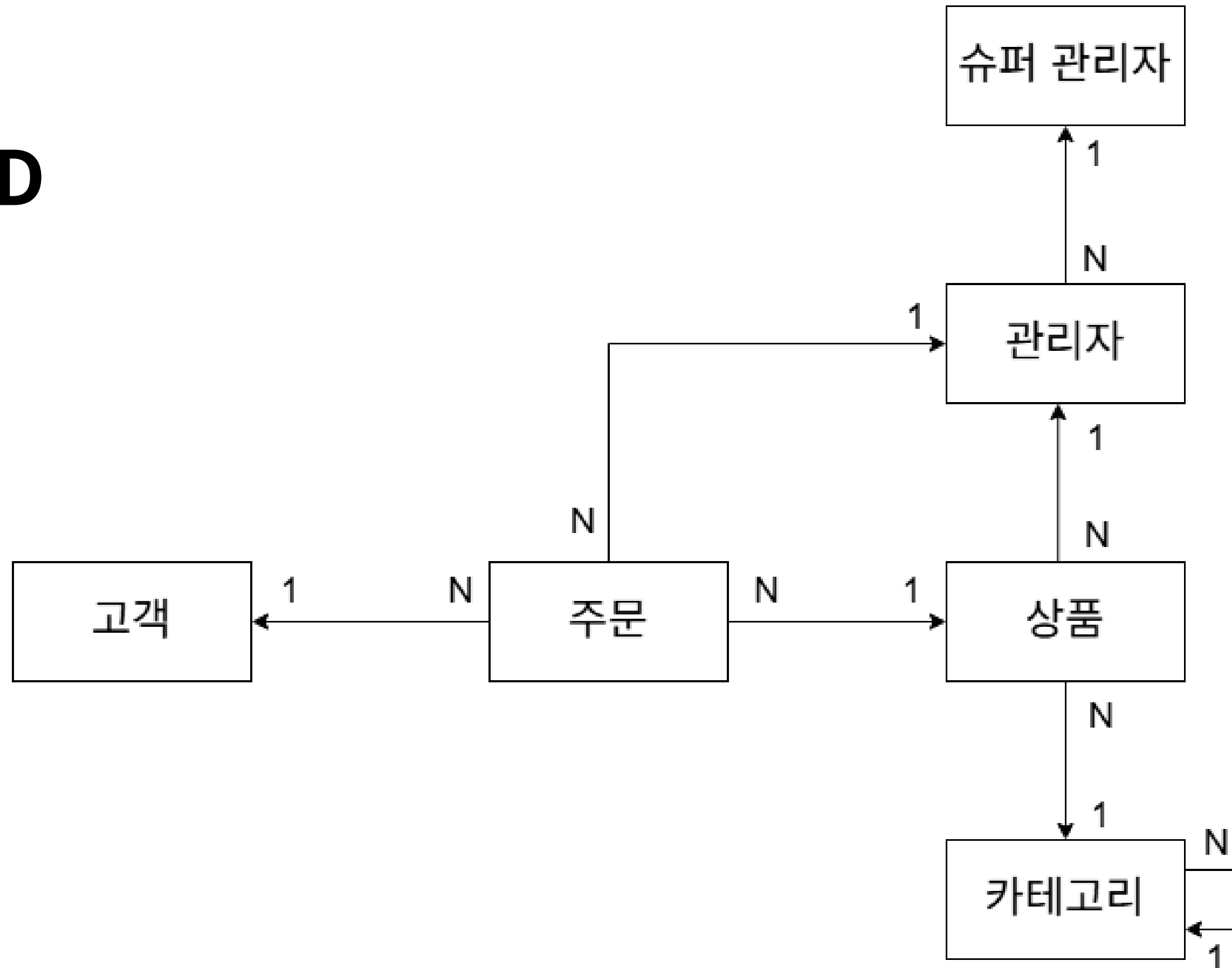
Architecture



- Session 기반 인증
- 공통 Response 포맷
- 전역 예외 처리

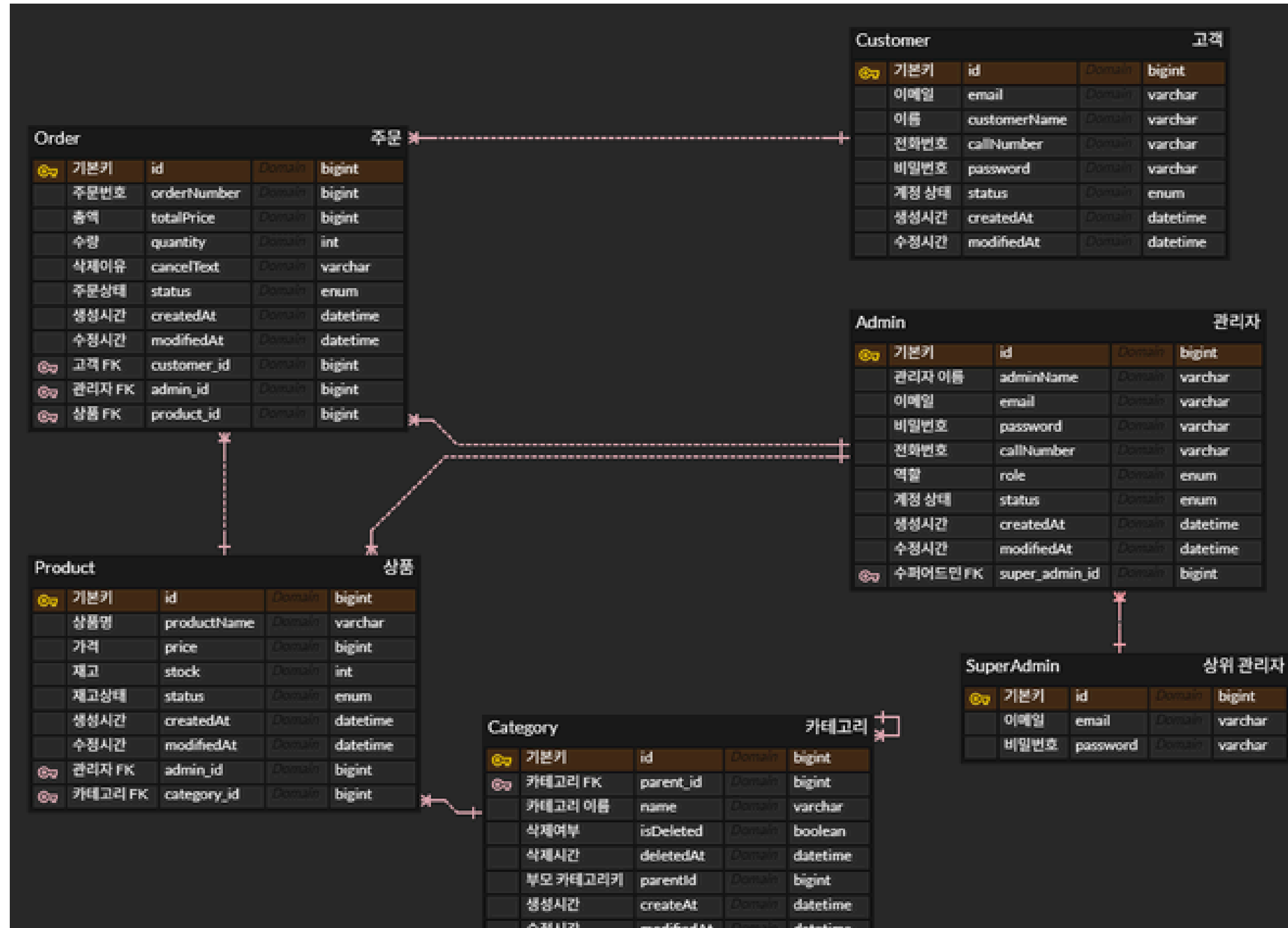
15

개념 ERD



16 시스템 설계(Architecture & ERD)

ERD



17

기술 스택

01

언어 & 프레임워크

자바 17

Spring boot 4.0.2

Lombok 1.18.42



Lombok



02

개발도구 & 테스트

Spring boot dev tools

Postman



POSTMAN



Gradle

03

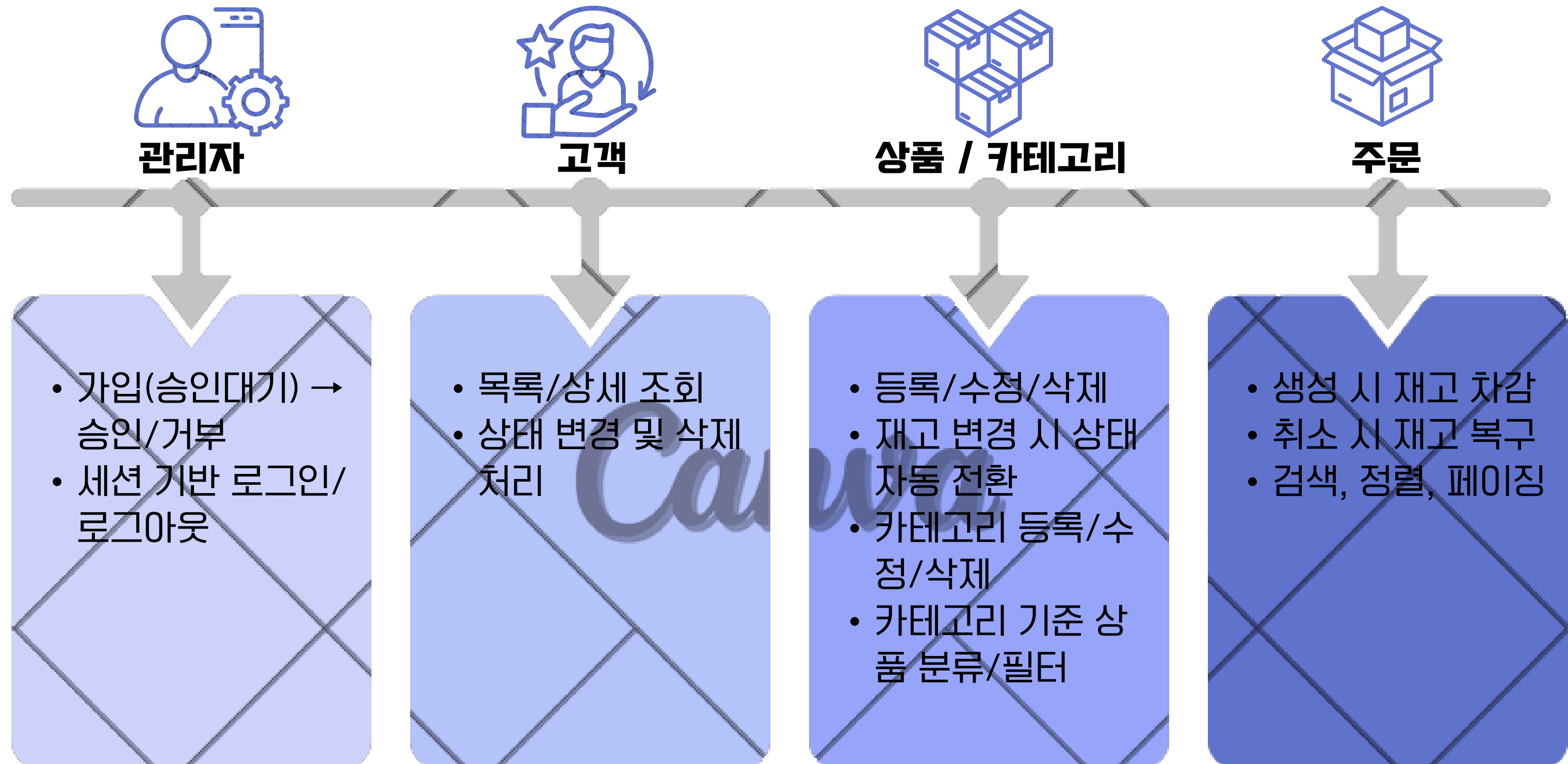
데이터 베이스

MySQL

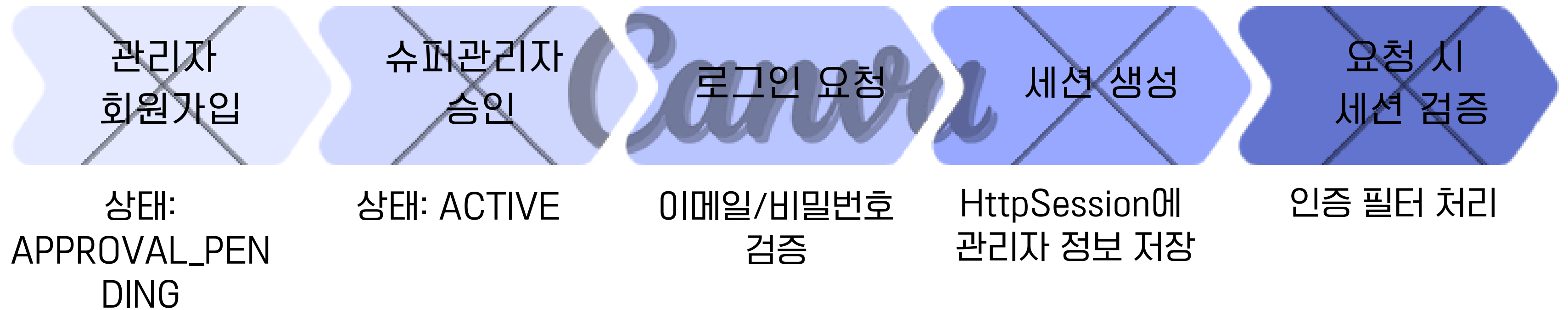
Spring Data JPA



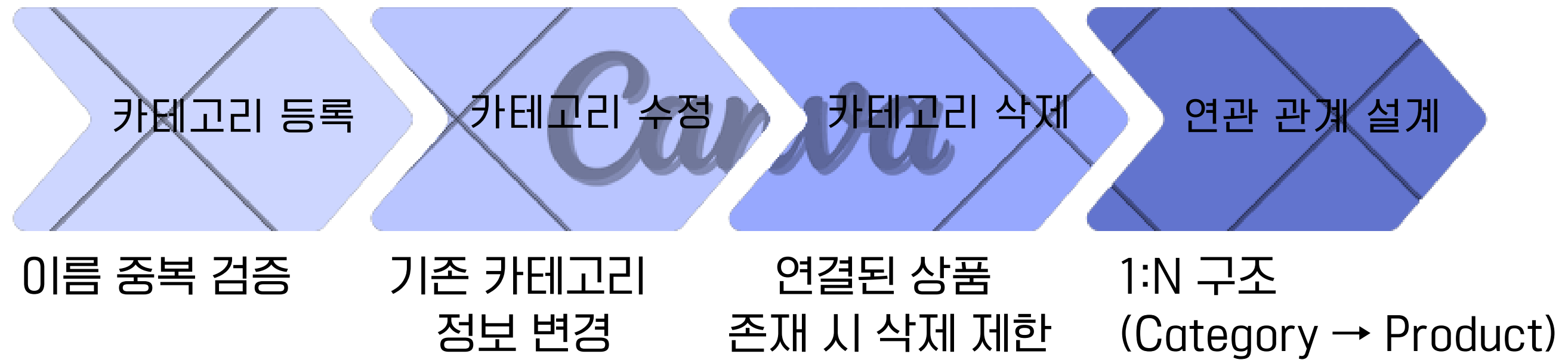
18 구현결과



19 관리자 로그인 흐름



20 카테고리 관리 흐름



21 상품 흐름



22 주문 처리 흐름



23 핵심 기능 시연



관리자 로그인
(Session 인증)



상품 등록 및
재고 변경



CS 주문 생성
(재고 자동 차감)



주문 취소
(재고 자동 복구)

24 트러블 슈팅

Soft Delete & Hard Delete

```
134 + Admin admin = adminRepository.findById(adminId)
135 + .orElseThrow(() -> new CustomException(ErrorCode.ADMIN_NOT_FOUND));
136 +
137 + adminRepository.delete(admin);
```



DuJoneKoo 3 days ago

Member ...

adminRepository.delete(admin); 을 하면 완전 삭제가 되니

admin.delete(); // status를 DELETED로 소프트딜리트로 변경하여
실수로 삭제하더라도 추적이나 복구 가능하게 하는건 어떠실까요?

문제 상황

문제 원인

해결 방법

결과 / 배운점

- adminRepository.delete(admin)사용 → 관리자 계정이 DB에서 완전 삭제(Hard Delete)됨
- 삭제 = 데이터 제거로 처리

- 관리자 계정 삭제는 “거래 데이터”는 아니어도 보안/운영 감사 관점에서 기록 보존이 중요
- 실수 삭제 시 복구/추적 불가
- 실수/권한 오남용 발생 시 근거 데이터가 사라짐

- 물리 삭제 대신 도메인 메서드로 위임
- admin.delete() → status = DELETED로 변경(Soft Delete) + 조회 시 제외

[결과]

- 삭제 이력 유지 및 복구 가능성 확보
- 정책 기반 삭제로 운영 안정성 강화

[배운점]

- 삭제는 단순 CRUD가 아닌 운영 정책의 영역
- 도메인에 책임을 위임하면 설계 일관성이 유지됨

25 트러블 슈팅

Soft Delete & Cascade 충돌

문제 상황

- Soft Delete 정책 적용 후 일부 연관관계에 cascade = REMOVE 가능성 존재
- 상위 엔티티 삭제 시 연관 엔티티가 물리 삭제될 위험

문제 원인

- Soft Delete는 “논리 삭제”
- Cascade REMOVE는 “물리 삭제 전파”
→ 삭제 정책이 서로 충돌
- 연관 데이터까지 실제 삭제될 경우 기록 보존 정책 무력화, 추적/감사 불가, 데이터 정합성 훼손 위험 있음

해결 방법

- Cascade 옵션으로 전체가 아닌, PERSIST, MERGE만 명시
- 삭제 책임을 Soft Delete에게 완전히 귀속

결과 / 배운점

[결과]

- Soft Delete 정책 일관성 유지
- 연관 데이터 보존으로 운영 안정성 강화

[배운점]

- Cascade는 편의 기능이 아니라 설계 리스크 요소
- 삭제에 대한 책임을 Soft Delete가 가질 수 있도록 넘길 수 있다.

26 트러블 슈팅

연관관계 설계(양방향 연관관계)

문제 상황

- 초기 설계에서 양방향 연관관계 및 셀프 조인을 고려
- 값을 수정했을 때, 연관 객체의 데이터 불일치 발생
- 엔티티 간 결합도 증가
- 삭제/변경 시 전파 영향 범위 확대

문제 원인

- 양방향 관계는 조회는 편하지만 변경 영향 범위 확대, 유지보수 복잡도 상승 문제 등 문제점 발견
- Soft Delete 정책과 결합 시 삭제 설계가 더 복잡해짐
- 양방향 셀프 조인 관계에서 조회를 수행하면서 N + 1 문제가 발생

해결 방법

- 편의 메서드를 통해 두 객체의 상태를 동기화하여 데이터 불일치 해결
- Fetch join을 활용해 N + 1 문제를 해결

결과 / 배운점

[결과]

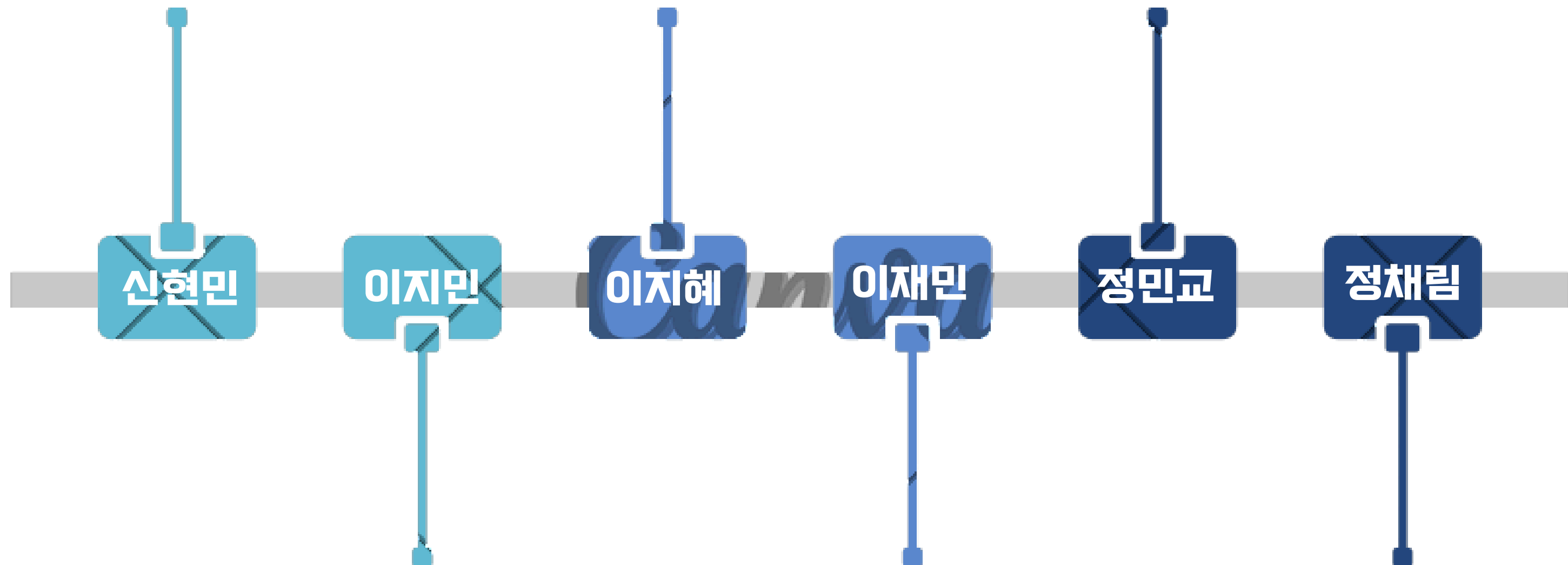
- N개의 추가 쿼리가 아닌, 조회 전용 1개 쿼리, 변경 쿼리 1개처럼 쿼리 수 감소
- 결합도 감소
- 유지보수성 향상

[배운점]

- 초기 설계는 단방향으로 설계하고, 정말 필요할 경우에만 양방향으로 리팩토링하기
- N + 1 문제는 Fetch Join 전략과 Batch Size로 제한하기

27 회고

- DTO를 활용한 가독성 향상을 경험
- 속도에 맞춘 원활한 소통과 기술적 기본 구성에 주력할 예정
- 패키지 및 기능 구현 경험에 만족
- Git 조작에 주의하겠다 다짐
- 명확한 서비스 책임 분리 구조에 만족
- 역할 분담을 잘 해야겠다 다짐



- 레이어 책임 분리와 코드 개선에 만족
- 설계 단계에서 도메인 책임과 세션 정의를 더 깊이 고민

- 팀원 간 소통과 배려에 만족
- 컨벤션을 잘 지켜야겠다 다짐



이상으로 발표를 마치겠습니다
궁금한 점이 있으시면 편하게 질문해주세요

THANK YOU

스파르타 내일배움캠프 Spring 3기 4조. 결제는 제가 하겠습니다

정채림, 신현민, 정민교, 이재민, 이지민, 이지혜